Power of Two :-

(16) $1 \times 2 = 2 \times 2 = 4 \times 2 = 8 \times 2 = 16$

$\frac{16}{2} = \frac{8}{2} = \frac{4}{2} = \frac{2}{2} = (1)$

$5/2 = 2$

$5.0/2 = 2.5$

$5/2.0 = 2.5$

$5.0/2.0 = 2.5$

int n = 2.5;

(n = 2)

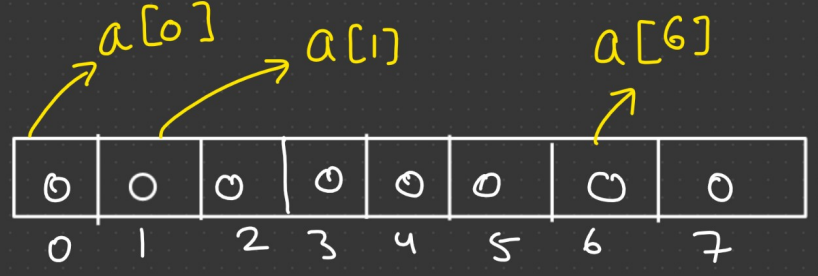
```
boolean isPowerofTwo ( double n)
{
    if ( n == 1)
        return true;
    else if (n > 1)
        return isPowerofTwo (n/2.0);

}
```

```
StringBuilder word = new StringBuilder("a");

Void getword(int k)
{
    if (k <= 1)
        return;
    else
    {
        StringBuilder temp= new StringBuilder();
        int len = word.length(); int i=0;
        while ( i<len)
        { if( word.charAt(i) == 'z')
                temp.append('a');
          else
            temp.append((char)(word.charAt(i) + 1));

            i++ ; k--;
        }
        word.append(temp);
        getword(k);
    }
```

Array :-

Array is collection of similar type of data element.

1D Array :-




```
int a[];  ⟶ Declare
a = new int[10]; ⟶ Assign
```

⟶

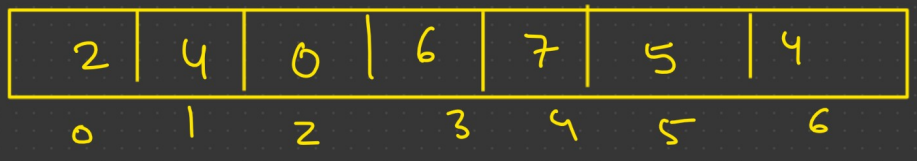
```
int a[] = new int[10];
```

initialization

Default value is zero.

```
a[0] = 1;
a[1] = 4;
a[2] = 6;
```

```
int a[] = { 2,4,0,6,7, 5,4 };
```

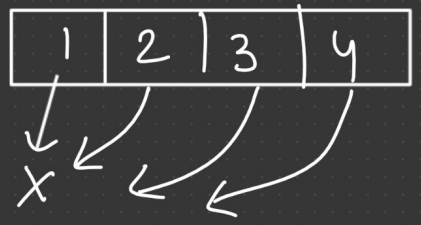


①

```
for( int i=0; i < a.length ;i++)
{
    System.out.println( a[i]);
}
```

②

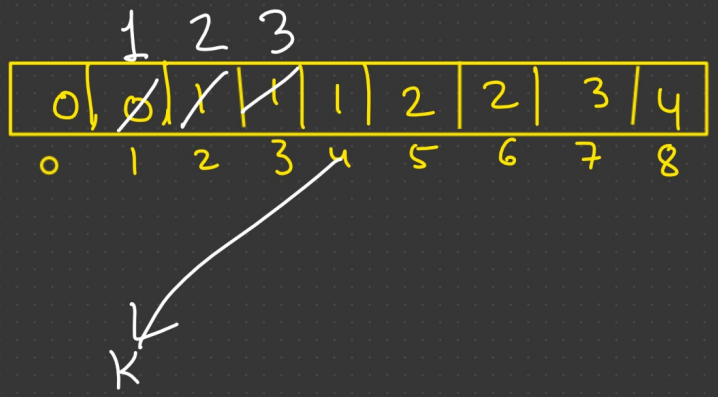
```
for ( int x : a)
{
    System.out.println(x);

}
```



Remove duplicate element:-

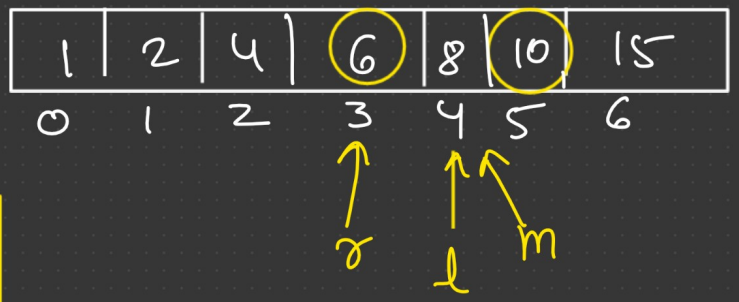
i = 1 2 3 4 5 6 7 8

```
if (a[i] != a[i-1])
{
    a[k] = a[i];
    k++;
}
i++;
```



Search element in sorted array :-

T = 7



$$mid = \frac{left + right}{2}$$

$$mid = \frac{0 + 6}{2} = 3$$

l = 0
R = 6

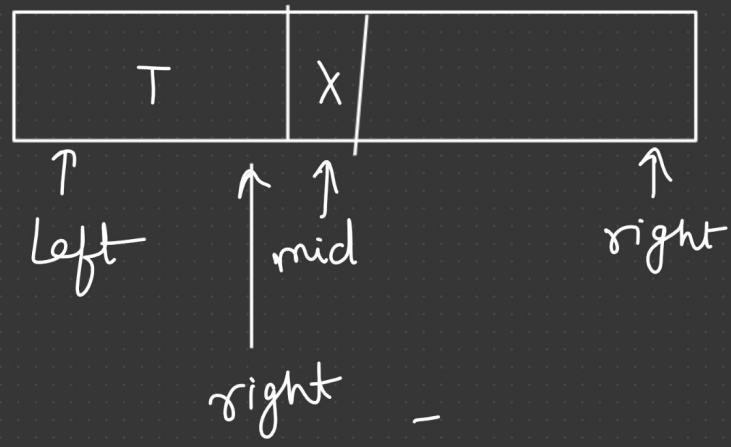
$$m = \frac{0+6}{2} = 3$$

l = 4
r = 6

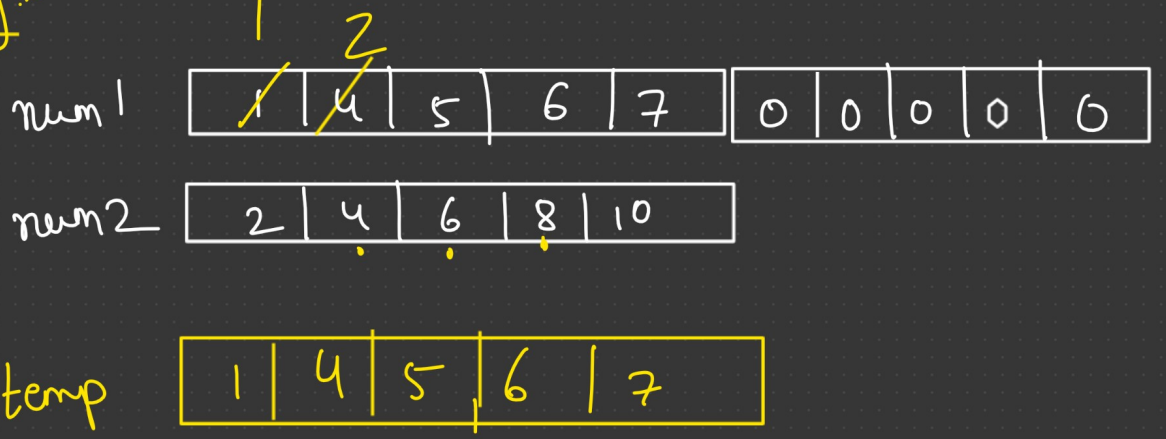
$$m = \frac{4+6}{2} = 5$$

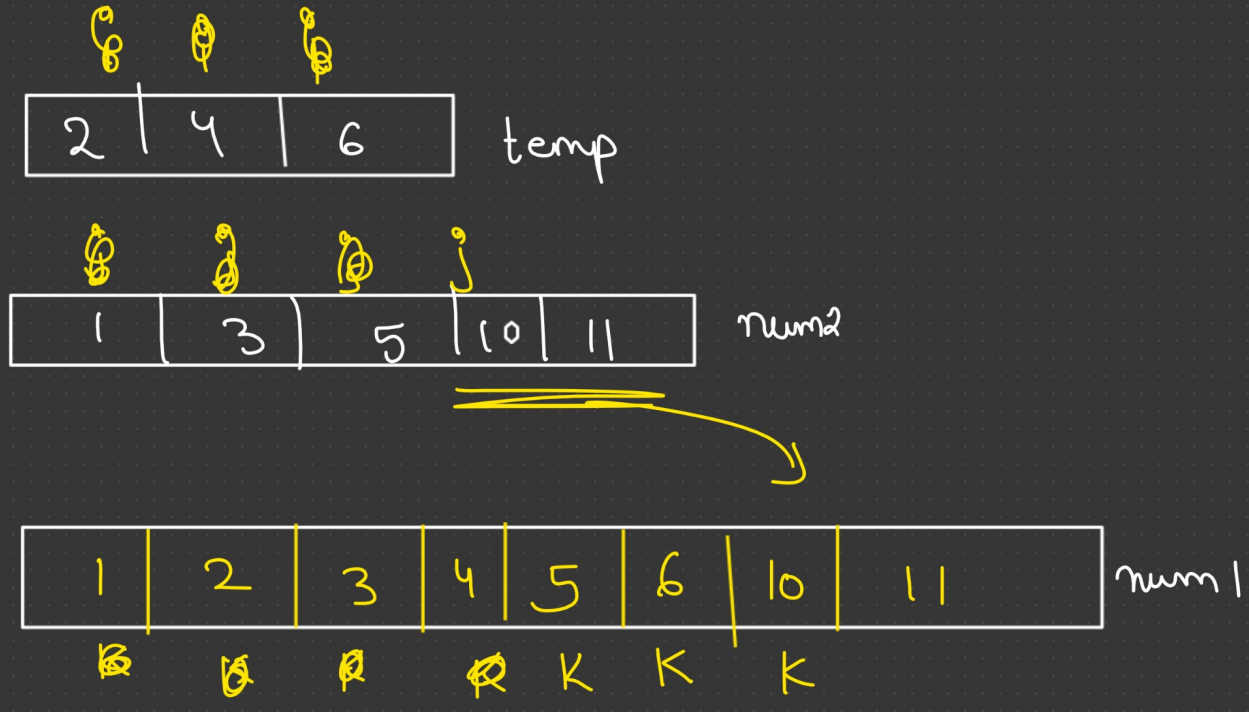
return 4;

```
int left=0; right = a.length-1;

while ( left <= right)
{
    int mid = (left+right)/2;

    if ( a[mid] == target)
        return mid;
    else if( a[mid] < target)
        left = mid+1;
    else
        right = mid-1;
}
return left;
```



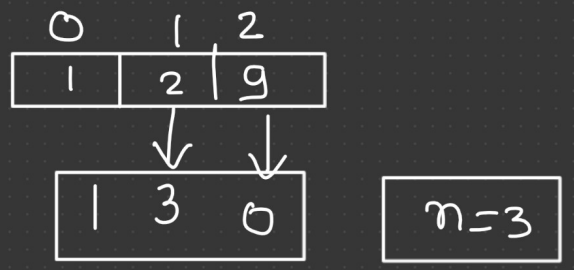
Merge Sorted Array :-

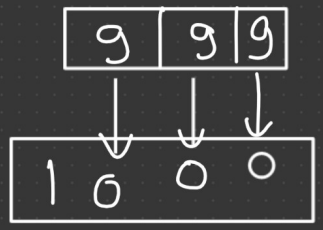




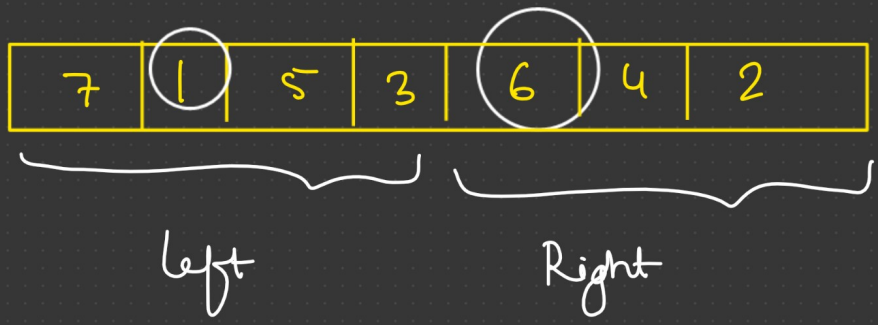
Plus One:-

```
for(int i = n-1 ; i >= 0 ; i--)
{
    if ( digit[i] < 9)
    {
        digit[i]++;
        return digit;
    }
    digit[i] = 0;
}
int temp[] = new int[n+1];
    temp[0] = 1;
return temp;
```



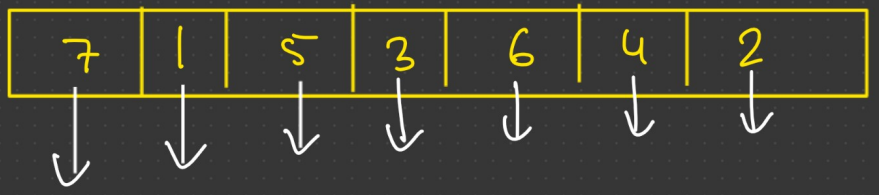


max profit in Stocks:-



(1)

return max { Left , Right , 6 − 1 = 5 }

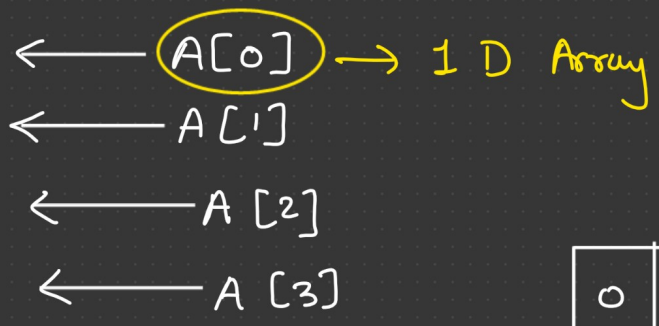


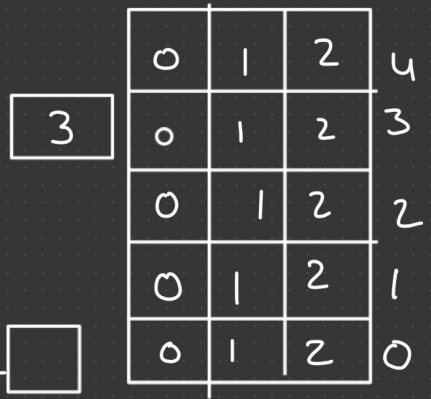
min = ~~1~~ 1

profit = 5 − 1 = 4
3 − 1 = ~~2~~
6 − 1 = 5
4 − 1 = ~~3~~
2 − 1 = ~~1~~

```
int profit = 0; int buy = Integer.Max_Value;

for (int x : prices)
{
    if (x < buy)
    {
        buy = x;
    }
    else{
        int t= x-buy;
        if(t > profit) profit=t;
    }

    return profit;
```

2-D Array in Java :-



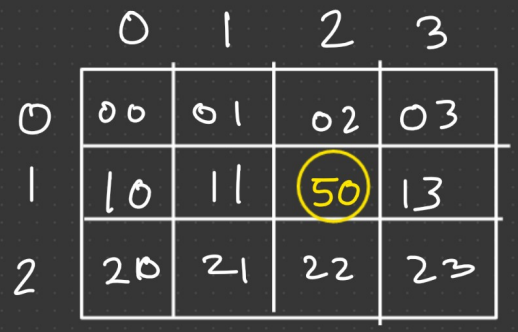


A[2][3] = 14

row col.

Building[3][0]

1 D Array element lift




```
int a[][] = new int[3][4];
```

3 one D - Array

4 elements in each 1D array

→ create 2D array

a[1][2] = 50

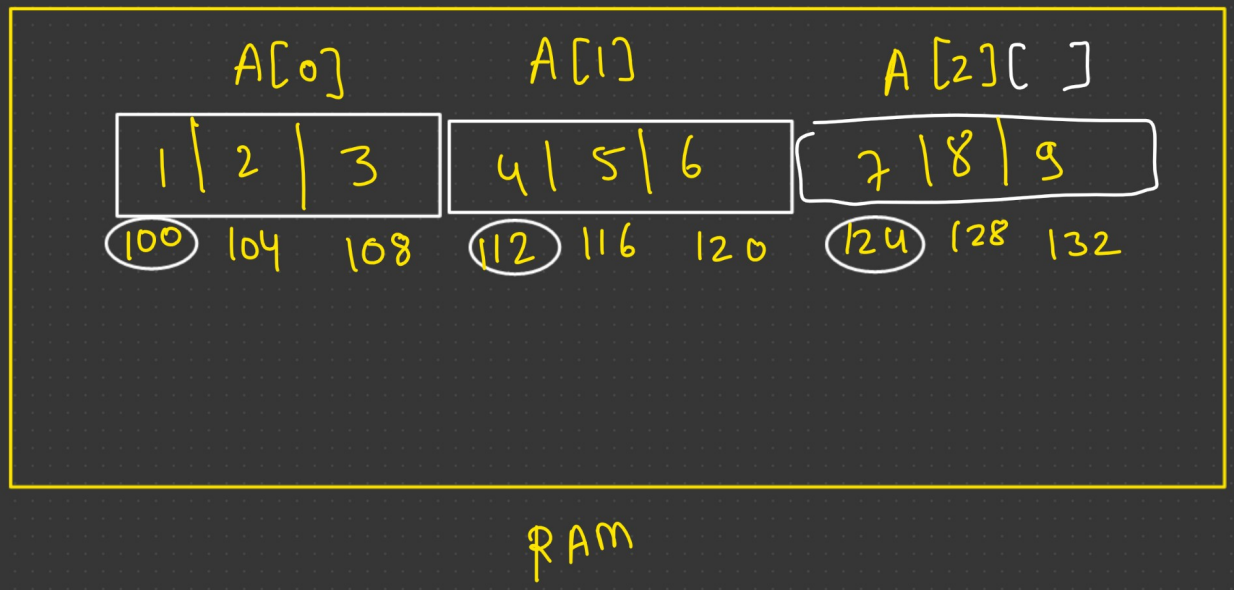
Total = 3 × 4 = 12

```
for(int i=0; i< a.length; i++)
{
    for (int j=0 ; a[i].length > j ; j++)
    {
        a[i][j]= 2;
    }
}
```

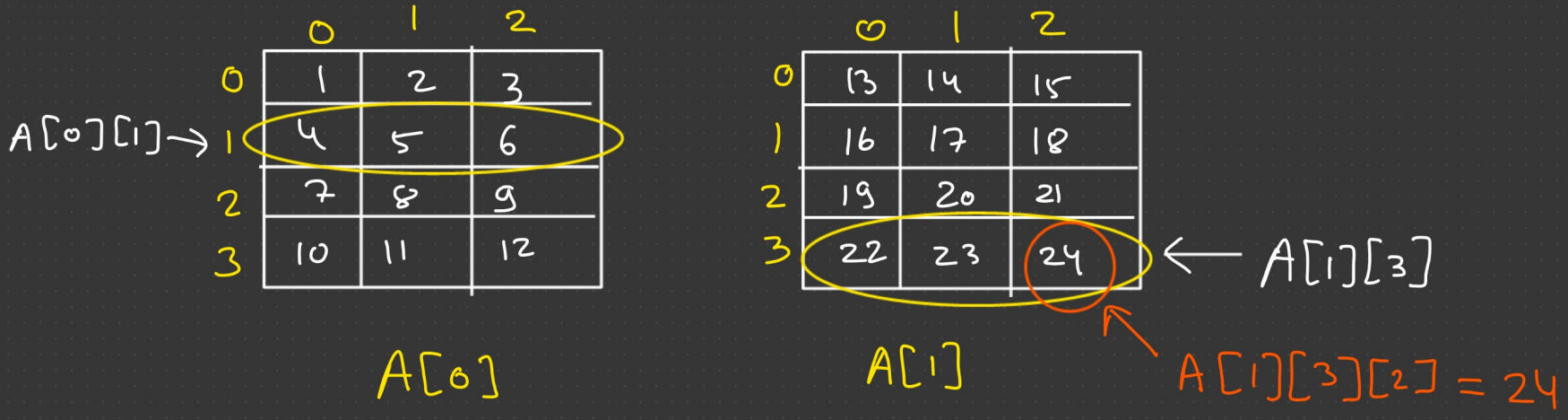
no. rows

no. of elements

$O(n^2)$



3D - Array



	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

	0	1	2
0	13	14	15
1	16	17	18
2	19	20	21
3	22	23	24

```
int a[][][] = new int[2][4][3];
```

- `2` → No of 2D Array
- `4` → No. of rows in each 2D array
- `3` → No. of elements in each row

Total = $2 \times 4 \times 3 = 24$

```
for(int i=0; i<a.length; i++)
{
  for(int j=0; j<a[i].length; j++)
  {
    for(int k=0; k<a[i][j].length; k++)
    {
      a[i][j][k] = 2;
    }
  }
}
```

No. of 2D array

No. of rows in each 2D Array

No. of elements in each row

$O(n^3)$

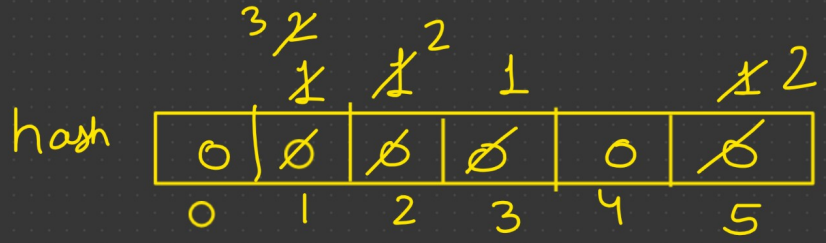
majority element:-

m = 2

count = 0 1

1	2	1	2	3	5	5	1

(1)



O(n)

1 → 3

2 → 2

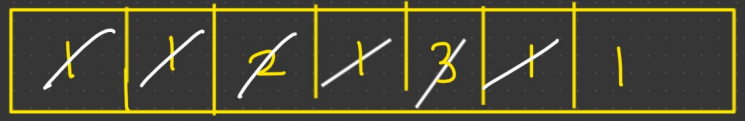
3 → 1

5 → 2

(2) sorting:-

1, 1, 1, 2, 2, 3, 5, 5

O(n log n)



result = 1

Count = 1 2 1 2

3 2 1

result = 1 2 ✓

Count = 1 2 1 0 1 2 1